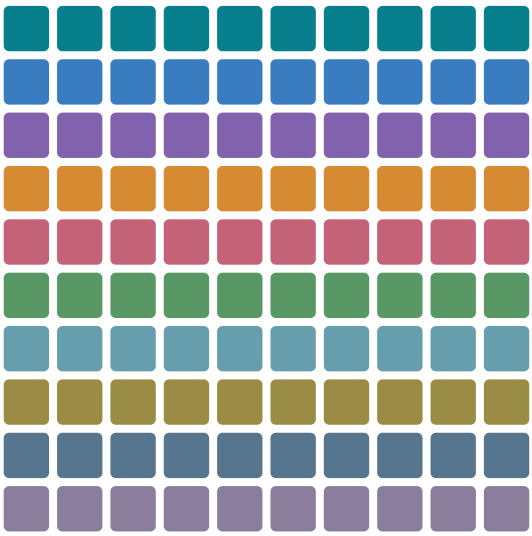
高并发场景，如何模拟？

以 100 个虚拟用户为例：看清谁在发请求、谁在排队、谁占用模型，以及如何判断测试结果。

真实历史数据　可控模拟假设　待实现设计



每个方块 = 1 个用户

100

虚拟用户

每人拥有独立身份、会话和请求 UUID。可由异步压测程序发起，无需手工打开 100 个浏览器窗口。

先把三个数字分开

100

在线 / 在途用户

客户侧连接与请求数量

10

模型执行槽示例

服务侧真实同时调用上限

90

首次排队示例

单调用场景中其余请求等待

这本图解的阅读顺序

第2-3页：准备用户与租户；第4-6页：选择负载、观察排队与公平；第7页：看真实历史结果；第8页：按步骤复现。

10 个槽与 90 个排队是教学假设，尚未作为本项目全局调度配置上线。

用一台发压机模拟 100 位客户

先准备用户与合成证据，再同步释放请求；服务端指标必须独立采集。

01 预置测试数据

每租户准备专属新闻/报告标记；每用户独立会话。记录版本和请求ID，保留原业务历史。

↓

02 等待起跑屏障

创建100个异步用户任务，全部等待同一Event；会话创建不计入消息突发延迟。

↓

03 同时放行 / 按速率放行

屏障一次放行测突发；定时按到达率放行测持续负载。记录每个请求的起止、状态和工具结果。

↓

后续双API验收拓扑 / HTTP与SSE链路

可信测试网关

验证身份与租户成员关系
剥离伪造角色头

→

API A / API B

鉴权、会话单飞
Python Agent循环

→

共享依赖与观测

PG事实源、模型接口
工具 / Temporal
采集资源占用

两种测试，分别得出结论

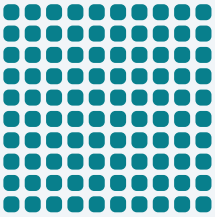
可控本地模型Port：固定延迟/错误，验证配额、取消、幂等。真实模型端点：验证实际容量与质量，消耗接口额度。

多副本用于验证全局额度不会翻倍。当前压测脚本覆盖单租户HTTP/SSE；测试网关、多租户夹具和共享调度仍待补齐。

同样 100 人，换四种分组

租户是组织/团队边界，用户是成员。不同布局回答不同问题，不能只替换一个 Header 就算完整验收。

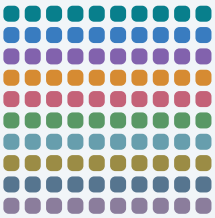
1 租户 × 100 用户



单租户基线

是否接住突发、排队是否有界。

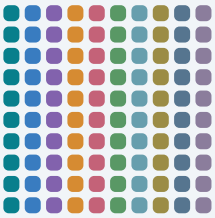
10 租户 × 10 用户



均衡竞争

每租户等待、份额和证据均正确。

100 租户 × 1 用户

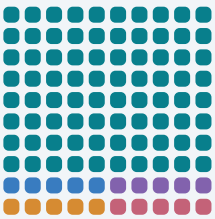


大量小租户

每个用户属于独立租户；检查无饥饿。

本图重复用色；100个方块各属不同租户。

80 + 4 × 5 用户



大租户施压

小租户能继续获得服务。

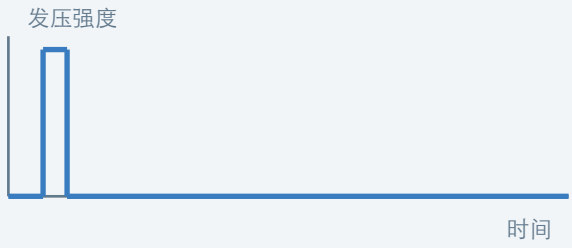
隔离检查要有可识别的数据

给A租户新闻加A专属标记，给B加B标记；同时查询，返回内容和模型上下文都只能出现当前授权作用域的证据。空结果不能代替正确取数验证。

选一种发压方式，再看一组指标

下面曲线是负载示意。虚拟用户数与请求到达率分别配置；慢响应不能自动降低目标到达率。

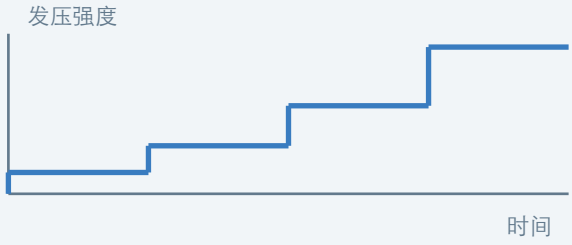
突发：100人同时点击



测瞬间受理、排队峰值与超时

用起跑屏障释放100条消息。

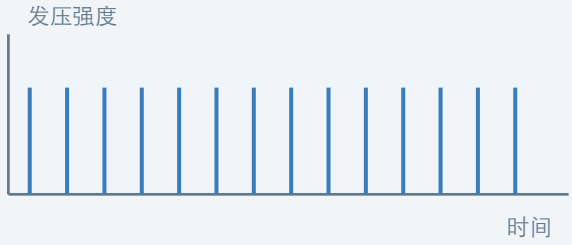
梯度：10 / 25 / 50 / 100人



每档稳定观察，再升下一档

分别记录每档完成率与P95；遇到错误先定位。

持续：固定到达率，15-30分钟



测队列增长、资源泄漏和恢复

按目标速率产生请求；设置最大在途数，超出也计入拒绝统计。

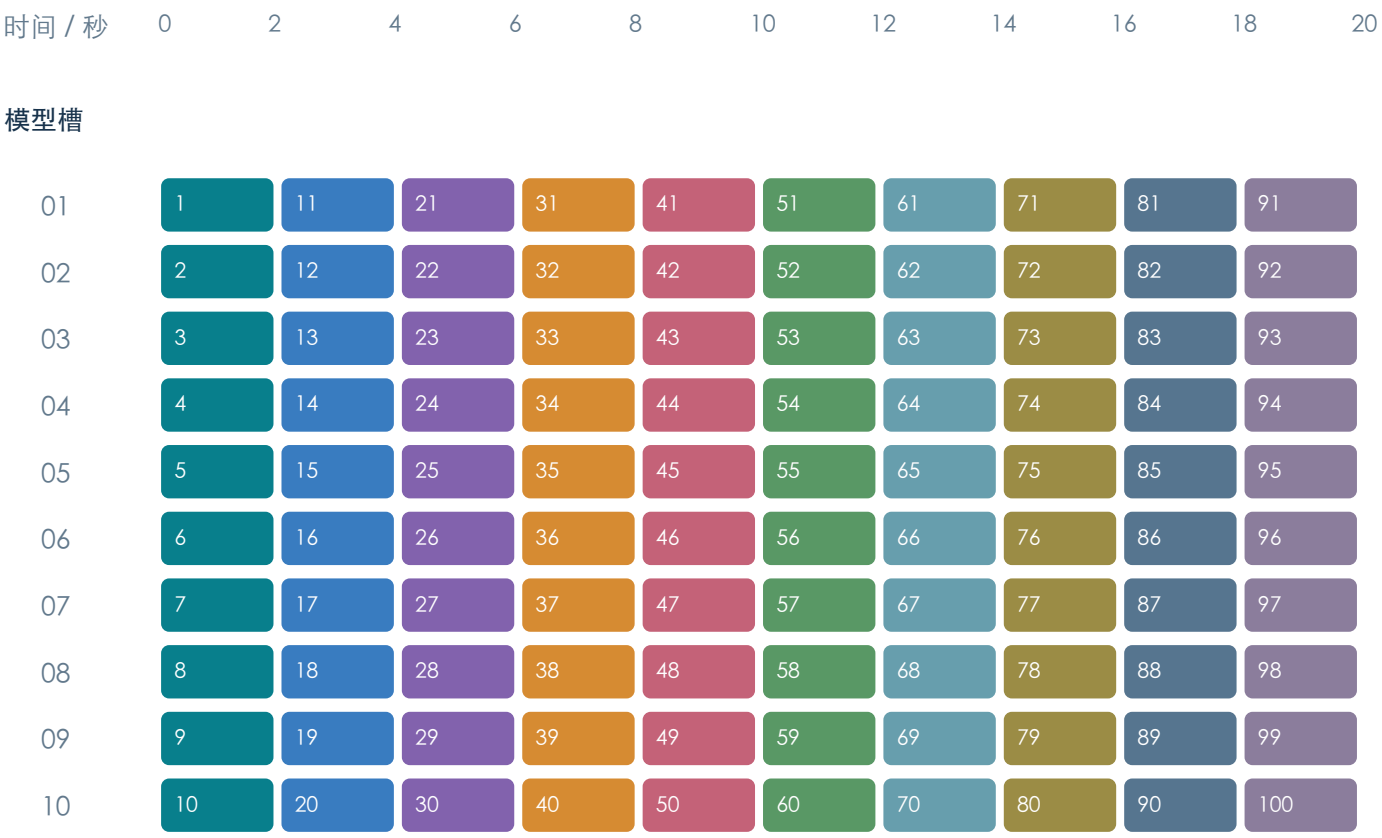
最容易误读的两个数

100个在途用户不是100 RPS；SSE首个进度事件不是有效答复。成功/失败/拒绝分别统计，同时给出每租户队列等待和端到端延迟。

已有 --rounds 是多次同步突发。固定到达率、持续时长和多租户分布参数尚未进入现有脚本。

100 个请求，10 个执行槽

教学假设：每请求只调用模型一次，每次固定2秒；全局10槽、无失败和额外开销。



格内数字 = 请求编号；每列完成10个请求。

首批开始时

10个执行中

90个等待

容量计算

10槽 ÷ 2秒 = 5次调用/秒
100次调用约需20秒

持续发压会怎样？

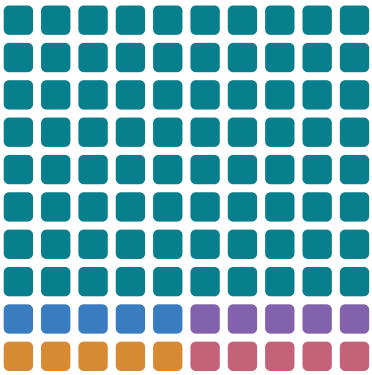
若持续8次/秒，饱和后每秒
新增约3个排队请求。

真实 Agent 一轮可能调用模型多次

规划时按模型调用数、token和工具耗时计算。模型槽只覆盖一次模型调用；等待SQL、Workflow或退避时归还。此图不能直接预测真实聊天的总耗时。

大租户施压，小租户还能前进

模拟：租户A有80人，B/C/D/E各5人；请求成本相同、租户等权、模型总窗口10。



- A · 80人
- B · 5人
- C · 5人
- D · 5人
- E · 5人

观察小租户的首次派发时间、等待P95和成功率，不能只看总吞吐。

如果A先排满共享FIFO



首批可能全是A，后来的小租户等待拉长。

按租户轮转，再选该租户用户



示意首批为A/B/C/D/E各2次；后续按有需求的租户继续轮转。

硬上限、权重、借用各管一件事

硬上限限制最大占用；权重决定竞争份额；其他租户空闲时可借用额度，仍不突破硬上限。本文示意的租户上限不成为瓶颈，也不静态预占每租户2槽。

以上是待实现的共享公平调度设计。跨2个以上API/Worker测试总额度，不能每个进程各建10槽。取消后远端调用未知时，保留保守占用。

历史实测：连接成功，业务仍会失败

2026-10-09归档；单租户100用户，真实glm-5.3-flash，普通对话与只读已有热点各占一半。

历史实测 / 非本次新压测

100个SSE连接均HTTP 200

66个completed

34个业务失败

completed且无失败/拒绝工具；失败码为model_unavailable。

指标	10用户基线	100用户突发
完成 / 尝试	10 / 10	66 / 100
首个SSE进度P95	0.032秒	0.794秒
成功轮次总耗时P95	10.108秒	11.097秒
历史一致 / 终态重放	各10 / 10	各100 / 100
其他用户 / 租户读被拒绝	各10 / 10	各100 / 100

报告至少输出这些维度

尝试、受理、拒绝、排队过期、业务成功、工具目标达成；总体与每租户队列等待/P95；服务端模型许可峰值、HTTP错误分类、DB等待和内存。

34个model_unavailable不能直接归因为429；历史通过包含失败轮次。跨租户读取被拒绝是越权检查，尚未验证多租户公平性。

来源：evaluation/results/conversation-load-{10,100}-existing-reports-20261009.json；均位于agent/。

按这条路线准备下一次测试

先核对依赖、数据与模型类型，再运行已有演示；多租户调度测试按设计阶段逐步补齐。

在agent/目录，使用已安装项目依赖的Python

A. 无网络模拟：单次运行内的新闻批量并发

```
python -m examples.hot_news_concurrency_demo
  --concurrency 3 --delay-ms 200
```

B. 单租户客户压测：先10用户，确认后再100

```
python tools/conversation_load.py --users 10
  --transport sse --workload mixed
  --output evaluation/results/load-10.json
```

上方命令分行展示参数，运行时各命令合为一行。B默认连接127.0.0.1:28000隔离演示API，可能调用真实计费模型；--users改为100时另选输出文件。

从现有脚本，到真正多租户模拟

01

准备身份与证据

统一各API鉴权；各租户合成canary；记录镜像/模型/策略版本。

02

实现共享准入与配额

queued独立于processing；PG原子绑定；同会话最多一个活跃请求。

03

补充发压与故障场景

10×10、100×1、80+4×5；至少2副本；取消、重放、429、崩溃。

生成链、校验与资料入口

归档JSON + 设计/工具文档 → load_sources()核对统计 → ConcurrencyGuide / ReportLab绘图 → 嵌入中文字体的8页PDF。参数：--output / --font / --bold-font。布局验收：Poppler逐页渲染、文字与页数检查。

资料：agent/docs/下的multi-tenant-concurrency-design.md、conversation-concurrency.md、hot-news-concurrency.md。TODO：共享调度/多租户夹具/真实配额与SLO验收。